Primer proyecto estructura de datos #1

Integrantes

Kristel Peraza Sánchez

Derek García Herra

Marco Cabrera Mora

Universidad Nacional de Costa Rica

Curso

EIF 207 – Estructura de datos

Profesor

Jose Calvo Suarez

Fecha de entrega

18 de octubre de 2026

Documento de Decisiones de Diseño

Este documento sigue el §8 del enunciado: tiene una sección por cada problema de estructuras de datos del §4, en el mismo orden. Cada sección indica quién la desarrolló, aunque los tres integrantes deben poder explicar cualquiera de ellas (§9).

Cómo se midió: las tablas de evidencia salen de `python main.py --bench`, que ejecuta cargas sintéticas generadas en el propio programa, sin usar la red ni datos de una cripta específica. Se reporta la **mediana** después de una ejecución de calentamiento, en milisegundos o, cuando se indica, en microsegundos por operación. Las mediciones se tomaron en Windows 11 con Python 3.14.7 el 18/10/2026. Cuando una sección cuenta operaciones reales, los conteos salen de reproducir con `--replay` partidas jugadas en las criptas del servicio web.

Principios comunes

- **Arquitectura por capas.** Datos, Lógica, Servicios e Interfaz, con DTO entre capas. La Interfaz nunca llama a Datos ni a Lógica; siempre pasa por Servicios (§6).
- **Estructuras propias.** `list` solo como almacenamiento contiguo; `dict` nunca como índice, caché ni tabla de búsqueda; sin `sort()`, `sorted()`, `heapq` ni `deque` (§7).
- **DTO inmutables y sin lógica**, con `@dataclass(frozen=True)` y solo métodos de conversión.
- **Estructuras compartidas en Estructuras/.** Solo va allí lo que usan dos o más capas: `ListaDoble`, `Ordenador`, `BufferCircular` y `Cola`.
- **Nada de una cripta concreta en el código** (§9): ni ids, ni llaves, ni capacidades, ni salas. Todo sale de los datos de la cripta.

Contenido

Sección	Problema	Responsable
§4.1	Qué ocurre a continuación (agenda de eventos)	Integrante 1
§4.2	Traer el contenido antes de que haga falta	Integrante 2
§4.3	Resolución de referencias del catálogo	Integrante 2
§4.4	Reutilización y caché persistente	Integrante 2
§4.5	Inventario	Integrante 3
§4.6	Pergaminos de retroceso e historial reversible	Integrante 3
§4.7	El rastro del jugador	Integrante 1
§4.8	Bitácora en pantalla	Integrante 3
§4.9	Ordenamiento	Integrante 3
§4.10	Guardado binario de la partida	Integrante 2
§4.11	Bitácora de partida y reproducción	Integrante 2
Anexo A	Otras decisiones de la capa de Lógica	Integrante 1

Sección	Problema	Responsable
Anexo B	Decisiones de Servicios, Controlador y Vista	Integrante 3
Anexo C	Reglas que el enunciado no define	Integrante 3

§4.1 Qué ocurre a continuación

Responsable: Integrante 1 — Marco Cabrera Mora (capa de Lógica)

1. Problema

En cada paso del bucle principal hay que obtener el evento pendiente con menor `tiempo_siguiente` y, si empatan, con menor número de secuencia. Operaciones: **programar** (cada acción de un actor programa la siguiente; también efectos, cierres de puertas y rearmes de trampas), **sacar el siguiente** (en cada paso), **cancelar** (un actor que muere, un antídoto) y **reprogramar** (un cambio de velocidad, §2.3). Frecuencia medida con `--bench`: entre dos decisiones del jugador se procesan unos 12 eventos con 10 enemigos activos, unos 105 con 100 y unos 521 con 500.

2. Alternativas consideradas

- **Array sin ordenar, buscando el mínimo:** sacar cuesta O(n) porque hay que revisar todos los eventos. Descartada: el §4.1 prohíbe examinar todos los eventos programados.
- **Array ordenado:** sacar el siguiente es O(1), pero programar es O(n) porque hay que correr elementos. Descartada; se mide directamente en `--bench`.
- **heapq o PriorityQueue:** prohibidos por el §7. Además, `PriorityQueue` está pensada para hilos y no permite cancelar eventos.
- **Montículo binario mínimo sobre un array (elegida):** programar y sacar cuestan O(log n).

3. Decisión

`Logic/ColaEventos.py` y `Logic/EventoPorg.py`:

- **Montículo binario mínimo sobre un array.** Programar y sacar el siguiente evento cuesta O(log n) y nunca hay que recorrer toda la agenda (§4.1). Se descartó el array ordenado: sacar cuesta O(1), pero programar es O(n). BenchLogica mide las dos opciones como evidencia.
- **Implementación propia, sin heapq ni queue.** El §7 prohíbe reemplazar las estructuras con bibliotecas. Además, PriorityQueue está pensada para hilos y no permite cancelar eventos.
- **Desempate por secuencia asignada por la cola.** Cumple la regla del §2.3 (menor tiempo y, si empatan, el que se programó antes) y hace que la partida sea determinista.
- **Cancelación perezosa (valido = False).** Cancelar cuesta O(1) sin buscar el evento dentro del montículo. Los cancelados se descartan solos al llegar a la raíz.

- **El tiempo de un evento nunca se modifica.** Reprogramar es cancelar y crear un evento nuevo. Si se cambiara el tiempo, el montículo quedaría desordenado sin enterarse.
- **_subir y _bajar iterativos.** En Python un ciclo es más rápido que la recursión, y ayuda a cumplir el límite de 20 ms (§5.2).
- **EventoProgramado como clase normal, no dataclass.** Es mutable, se compara por identidad y tiene un orden propio. Con dataclass habría que desactivar casi todo lo que genera.
- **Integración con el retroceso:** cada evento programado y cada evento extraído se registran en el historial, para que el pergamino deje la agenda exactamente como estaba (§4.6).

4. Análisis de complejidad

Operación	Costo
Programar	O(log n): se agrega al final y sube
Sacar el siguiente	O(log n), más los eventos cancelados que estén en la raíz (cada cancelado se descarta una sola vez)
Cancelar	O(1): se marca como inválido, sin buscarlo
Reprogramar (cambio de velocidad)	O(log n): cancelar + programar uno nuevo con secuencia nueva

5. Evidencia empírica

Agenda elegida contra la descartada. Con n eventos en la agenda se programan 100 más y se sacan 100. Costo por evento (mediana):

Eventos en la agenda	Programar: heap / array	Sacar: heap / array
100	0.87 / 9.51 µs	1.78 / 0.11 µs
1 000	0.83 / 78.01 µs	3.49 / 0.11 µs
10 000	1.44 / 781.81 µs	4.92 / 0.11 µs

El array ordenado saca más rápido, pero en el juego casi cada evento que sale programa otro (un actor que actúa programa su próxima acción). Sumando las dos operaciones, con 100 eventos el heap cuesta 2.65 µs por evento contra 9.62 µs del array (3.6 veces menos), y con 10 000 eventos cuesta 6.36 µs contra 781.92 µs (unas 120 veces menos). El costo del heap casi no crece con el tamaño de la agenda; el del array crece de forma lineal.

Ciclo completo de eventos (§5.2). El jugador espera y el motor procesa todo hasta su siguiente decisión (200 ciclos seguidos, en una cuadrícula sintética con enemigos errantes y rastreadores):

Enemigos activos	Eventos por ciclo	Mediana	Peor ciclo	Límite de 20 ms
10	~12	0.07 ms	0.16 ms	Cumple
100	~105	0.82 ms	14.48 ms	Cumple
500	~521	4.35 ms	16.92 ms	Cumple

6. Límites conocidos

- Con la cancelación perezosa, los eventos cancelados ocupan espacio hasta llegar a la raíz. Con muchísimas cancelaciones (por ejemplo, cambios de velocidad muy seguidos) el montículo crece y sacar descarta varios eventos seguidos.
- El peor ciclo varía entre corridas: la mediana es estable, pero los picos dependen de pausas del sistema operativo o del recolector de basura de Python. En una corrida anterior, con 500 enemigos, un ciclo llegó a 119 ms aunque la mediana fue de 9.3 ms. Con la cripta de referencia, que tiene muchos menos enemigos activos a la vez, el margen es amplio.
- El montículo no permite recorrer los eventos en orden; para mostrar la agenda ordenada habría que ordenar una copia. El juego no lo necesita.

§4.2 Traer el contenido antes de que haga falta

Responsable: Integrante 2 — Derek García Herra (capa de Datos)

1. Problema

Antes de confirmar un movimiento, el contenido de la sala destino debe estar en memoria. Cada solicitud tarda ~400 ms, cuesta lo mismo con 1 que con 10 salas y cuenta contra el presupuesto. En cada movimiento el programa decide si debe pedir contenido cercano y comprueba que la sala destino esté cargada. En una partida real de `cripta-02` (31 movimientos) toda la partida usó **6 solicitudes de 23**.

2. Alternativas consideradas

- **Pedir cada sala al entrar:** ~400 ms de espera en cada puerta y una solicitud por sala (100 salas = 100 solicitudes). Descartada.
- **Pedir todo al inicio:** sin esperas, pero gasta [salas/10] solicitudes aunque el jugador visite pocas; en criptas grandes agota el presupuesto. Descartada.
- **Precarga por cercanía (elegida):** pedir solo cuando falta algo cerca y llenar cada solicitud con 10 salas.

3. Decisión

`Data/GestorContenido.py`: un array `_estado` indexado por id de sala (`NO_PEDIDA`, `PENDIENTE`, `CARGADA`) impide pedir dos veces la misma sala. Al llegar a una sala, un BFS limitado a **2 pasos** revisa si falta alguna; si falta, otro BFS toma las **10 salas sin pedir más cercanas** y hace una

sola solicitud. `asegurar_sala` garantiza la sala destino antes de confirmar el movimiento. El BFS recorre N, S, E, O y el contenido se incorpora en el orden pedido, así que es determinista; es síncrono y la espera real nunca avanza el reloj virtual. Si una precarga falla, el juego sigue.

4. Análisis de complejidad

- ¿Sala cargada?: **O(1)**.
- Decidir si pedir: BFS de 2 pasos, como máximo 17 salas sin importar el tamaño de la cripta (más un array de distancias O(S), muy barato).
- Armar un lote: **O(S + salidas)** en el peor caso, solo cuando se pide. Memoria: O(S).

5. Evidencia empírica

Cuadrícula sintética; el jugador recorre 100 salas vecinas:

Salas	Solicitudes (precarga)	Una por sala	Decisión (100 pasos)
100	10	100	1.50 ms
900	22	100	2.53 ms
3600	27	100	2.86 ms

Entre 4 y 10 veces menos solicitudes (de ~40 s de espera a 4–11 s), y la decisión casi no crece con el tamaño de la cripta (menos de 0.03 ms por paso con 3600 salas).

6. Límites conocidos

- Es síncrona: si la sala destino no se precargó (falla de red o presupuesto agotado), el jugador espera la respuesta antes de moverse.
- Con muchas salidas por sala, a 2 pasos puede haber más de 10 salas y harían falta dos solicitudes.
- Los arrays suponen ids de sala seguidos (1..N); el contenido no se guarda en disco entre ejecuciones.

§4.3 Resolución de referencias del catálogo

Responsable: Integrante 2 — Derek García Herra (capa de Datos)

1. Problema

El contenido trae solo el tipo de cada entidad (por ejemplo `ent_rata_gigante`), y antes de mostrarla o simularla hace falta su ficha. Los ids llegan en orden no controlado, se repiten (dos ratas, una ficha) y se comparten entre criptas. Se resuelven todos al llegar cada lote de contenido, y `ficha(id)` se consulta al crear entidades o armar el estado (34 búsquedas en la partida real de `cripta-02`).

2. Alternativas consideradas

- **Un id por solicitud:** 11 tipos = 11 solicitudes. Descartada (el §4.3 dice que no cumple el presupuesto).
- **Resolver al usar:** una solicitud y una espera de ~400 ms por cada tipo nuevo en plena partida. Descartada.
- **Resolver por lotes al llegar el contenido (elegida):** memoria → disco → red, sin repetidos, en lotes de 10.

3. Decisión

`Data/ResolvedorCatalogo.py` busca cada id en memoria (`CacheCatalogo`), luego en disco (`AlmacenCatalogo`). Lo que falta se encola en una `Cola` en el orden en que se descubre (determinista), un array `vistos` evita repetidos, y se pide en lotes de hasta 10. Todo lo descargado se guarda en disco, así que un id vigente nunca se pide dos veces, ni en otra ejecución ni en otra cripta. También se resuelve el campo `suelta` de los enemigos, para que su botín ya esté disponible al morir.

4. Análisis de complejidad

Con k ids por lote y m faltantes: quitar repetidos O(k²) (k son decenas); memoria **O(log n)** por id; disco una lectura por id; red **[m/10] solicitudes** en lugar de m.

5. Evidencia empírica

- `--bench`: 200 fichas fuera de memoria se recuperan del disco en 24.9 ms; desde la red serían 20 solicitudes en lotes, no 200.
- `cripta-01` tiene 11 tipos: 2 solicitudes de catálogo en lugar de 11.
- Partida real en `cripta-02`: 23 aciertos en memoria y 11 fallos resueltos desde el disco con fichas guardadas en partidas de `cripta-01`; ningún tipo ya conocido se volvió a descargar. (Esa partida se jugó antes de que las fichas descargadas por la precarga se contaran también como fallos; con fichas ya guardadas en disco, como en este caso, el conteo no cambia.)

6. Límites conocidos

- Quitar repetidos es cuadrático en el tamaño del lote; con cientos de tipos por lote convendría ordenarlos.
- Si una ficha no se resolvió de antemano, `ficha(id)` la pide en ese momento, con su espera.
- Un cambio de versión del catálogo invalida todo lo guardado.

§4.4 Reutilización y caché persistente

Responsable: Integrante 2 — Derek García Herra (capa de Datos)

1. Problema

Las fichas en memoria tienen un tope (`--cache-size`, 25 por defecto). Al llenarse hay que liberar una que no esté en uso; la liberada debe recuperarse del disco y no de la red, y la caché debe sobrevivir al cierre validándose con la versión del catálogo. La operación más frecuente es **buscar** (34 veces en la partida de `cripta-02`); insertar solo ocurre en fallos (11) y liberar solo al superar el tope.

2. Alternativas consideradas

- **Búsqueda:** array sin ordenar con búsqueda lineal O(n) (descartada y medida en `--bench`); `dict` (prohibido por el §7); árbol (no permitido por el profesor); **array ordenado con búsqueda binaria O(log n) (elegida)**.
- **Qué liberar:** FIFO puede sacar una ficha muy usada solo por ser antigua (descartada); **LRU (elegida)**: el jugador avanza por salas vecinas, así que lo usado hace poco es lo más probable de volver a necesitarse.
- **Disco:** un solo archivo con todo el catálogo obliga a leerlo o reescribirlo completo (descartada); **un archivo por ficha en una carpeta por versión (elegida)**.

3. Decisión

- **Memoria (Data/CacheCatalogo.py):** `_entradas`, array ordenado por id con la ficha y su contador de usos; `_recientes`, la `ListaDoble` del equipo con **solo las fichas liberables**, de la más reciente a la más antigua. Usar una ficha mueve su nodo al frente; liberar toma la cola. Una ficha en uso sale de la lista, así que nunca se libera. Si todas están en uso, la caché supera el tope temporalmente. Tras un pergamino la caché no se revierte; solo se rehacen las marcas de uso. Para el reporte final (§5.1), un **acierto** es una ficha encontrada en memoria y un **fallo**, una ficha que no estaba en memoria al consultarla o que hubo que descargar durante la precarga.
- **Disco (Data/AlmacenCatalogo.py):** `cache/catalogo/<version>/<id>.json`. Al iniciar se consulta `GET /catalogo/version` (una solicitud liviana): si coincide, todo lo guardado sirve; si no, se borra y se vuelve a pedir. Cada ficha se escribe a `.tmp` y luego reemplaza al archivo; un archivo dañado se descarta sin detener el juego.

4. Análisis de complejidad

Operación	Costo
Buscar	O(log n), búsqueda binaria
Acierto (actualizar recencia), elegir qué liberar	O(1), lista doble
Insertar o quitar del array	O(n), correr elementos (solo en fallos y desalojos)
Marcar / desmarcar en uso	O(log n)
Recuperar una ficha liberada	1 lectura de un archivo pequeño

5. Evidencia empírica

Búsqueda elegida contra la descartada (500 consultas, la mitad existen):

Fichas	Binaria (elegida)	Lineal (descartada)	Lineal / binaria
25 (por defecto)	0.26 ms	0.21 ms	0.8×
100	0.34 ms	0.71 ms	2.1×
1 000	0.53 ms	6.52 ms	12×
10 000	0.74 ms	61.96 ms	84×

Recuperar 200 fichas: memoria 0.21 ms, disco 24.9 ms (~0.12 ms por ficha), red 20 solicitudes de ~400 ms.

Con el tope por defecto la búsqueda lineal es un poco más rápida (0.05 ms en 500 consultas, despreciable). Se eligió la binaria porque el tope es configurable y el catálogo se comparte entre criptas: desde 100 fichas ya gana, y con 10 000 su costo casi no cambia mientras la lineal se multiplica por 300. El disco es ~100 veces más lento que la memoria, pero miles de veces más barato que la red y no gasta presupuesto.

6. Límites conocidos

- Insertar y quitar son O(n): con topes de cientos de miles y muchos fallos se notaría (la alternativa sería un árbol, no permitido).
- LRU no considera la frecuencia: alternar entre zonas lejanas puede liberar fichas que se volverán a usar (cuesta una lectura de disco).
- El disco no tiene tope, y un cambio de versión del catálogo descarta todo lo guardado.

§4.5 Inventario

Responsable: Integrante 3 — Kristel Peraza Sánchez

1. Problema

El jugador lleva una colección ordenada de objetos con capacidad máxima `inventario_max`, que fija cada cripta. Debe poder recorrerla hacia adelante y hacia atrás desde una posición actual (el cursor) y soltar el objeto que está viendo, en cualquier posición, sin que el costo dependa de dónde esté. Equipar un objeto lo mueve al frente del orden real. Además hay que mostrar vistas ordenadas por peso, valor o nombre sin alterar el orden real, y rechazar recoger cuando está lleno.

Operaciones que ejecuta el programa, contadas al reproducir una partida real de `cripta-03` (154 acciones, victoria):

Operación	Cuándo ocurre	Veces en la partida
Recorrer (siguiente / anterior)	Rueda del ratón o Tab	No queda en la bitácora; es la más frecuente al jugar
Recoger	Acción de costo 25 sobre un objeto del suelo	34
Ubicar por id	Al traducir una acción (jugador o replay) al objeto concreto	29
Soltar el objeto del cursor	Soltar, y también usar, que consume el objeto	27 (3 soltar y 24 usar)
Equipar	Acción de costo 50	2
Armar la vista del inventario	Una vez por acción, al entregar el estado a la Vista	155

2. Alternativas consideradas

- **Arreglo contiguo (list):** acceso por posición en O(1) y recorrido simple. Descartada: soltar o equipar en la posición i desplaza elementos, O(n), y el costo depende de la posición, lo que prohíbe el §4.5. Se conserva como referencia en `--bench`.
- **Un nodo propio solo para el inventario:** una lista doble hecha a la medida. Descartada: la caché del catálogo también necesita una lista doble para su política de desalojo, y habría dos implementaciones casi iguales.
- **Lista doblemente enlazada genérica (elegida):** insertar y quitar en O(1) teniendo el nodo, y recorrido en ambos sentidos.

3. Decisión

`Estructuras/ListaDoble.py` (con `NodoDoble`) y `Logic/Inventario.py`, construido encima:

- **El nodo como referencia.** Cada `agregar_*` devuelve su nodo. Soltar y equipar reciben el nodo y no recorren nada. El nodo recuerda a qué lista pertenece, lo que impide un doble borrado que descuadraría el contador.
- **Cursor y capacidad.** `Inventario` guarda el cursor y la capacidad máxima tomada de los datos de la cripta. Con el inventario lleno, recoger se rechaza sin consumir tiempo (Apéndice B).
- **Equipo marcado por nodo.** Con varios objetos del mismo id, la acción solo trae el id. El arma y la armadura equipadas se marcan por nodo, y `ubicar(id)` prefiere la instancia de repuesto. Así el replay es determinista sin cambiar el DTO, y soltar «una daga» no suelta la que se lleva equipada.
- **Vista ordenada.** `vista_ordenada(criterio)` entrega una copia ordenada con el `Ordenador` por peso, valor o nombre (sin distinguir mayúsculas). Los empates conservan el orden real y el orden real no cambia (§4.9).

- **Quién coordina.** `Inventario` no conoce la mazmorra ni al jugador. El Motor coordina recoger y soltar con el suelo, el costo en tiempo y el historial. `ServicioAccionJugador` ejecuta recoger, soltar, equipar y usar; `ServicioInventario` solo recorre y cambia el orden de la vista.
- **Reversible.** `Inventario` registra sus cambios en el historial para el pergamino de retroceso (§4.6).

Las reglas de comportamiento que el enunciado no fija (dónde se recoge un objeto, qué hace el cursor en los extremos, etc.) están en el Anexo C.

4. Análisis de complejidad

Sea n la cantidad de objetos en el inventario, con n ≤ `inventario_max`.

Operación	Costo	Comentario
Recoger	O(1)	Se agrega al final; la lista conoce su último nodo.
Soltar el objeto del cursor	O(1)	Se desenlaza el nodo; no depende de la posición.
Equipar	O(1)	Se desenlaza el nodo y se inserta al frente.
Siguiente / anterior	O(1)	El cursor avanza un nodo.
¿Está lleno?	O(1)	Se compara un contador con la capacidad.
Ubicar por id	O(n)	Búsqueda lineal; una vez por acción, y n lo acota la cripta.
Vista ordenada	O(n²) a O(n log n)	La decide el Ordenador (§4.9); no modifica el orden real.

Las operaciones frecuentes (recorrer, recoger, soltar y equipar) no dependen de n ni de la posición, que es lo que pide el §4.5. La única lineal es ubicar, y ocurre una vez por acción del jugador.

5. Evidencia empírica

Lista doble contra el arreglo descartado (grupo «inventario (estructura)» de `--bench`). Costo por operación, en microsegundos (mediana de tandas de 200 operaciones; el recorrido es uno completo):

Operación	n = 1 000	n = 10 000	n = 100 000
Quitar y reponer el del medio: lista / arreglo	0.31 / 0.32 µs	0.33 / 2.88 µs	0.32 / 29.5 µs
Mover el último al frente: lista / arreglo	0.30 / 0.53 µs	0.30 / 4.81 µs	0.30 / 48.7 µs
Recorrer todo: lista / arreglo	25 / 31 µs	257 / 337 µs	2 497 / 3 193 µs

Con la lista doble, quitar del medio y mover al frente cuestan lo mismo con mil que con cien mil objetos. En el arreglo crecen con n: con 100 000 objetos, quitar del medio es unas 90 veces más lento y mover al frente unas 160 veces. Con 1 000 objetos quitar del medio cuesta prácticamente igual en las dos, porque el arreglo de Python desplaza memoria muy rápido; la decisión no se apoya en el tamaño actual (`inventario_max` es 10 en las criptas del servicio), sino en que el §4.5 exige un costo independiente de la posición y cada cripta puede fijar otra capacidad.

La clase Inventario completa (grupo «inventario (clase)»): con 10 objetos, ubicar por id cuesta 0.37 µs y una vista ordenada por nombre 10 µs; con 1 000 objetos, ubicar cuesta 17 µs. Además, el comportamiento de `Inventario` se comparó contra un modelo de referencia sencillo con cientos de operaciones aleatorias.

6. Límites conocidos

- Con un `inventario_max` muy pequeño (unas pocas decenas), un arreglo sería más simple e igual de rápido, como muestra la medición con 1 000. La lista doble se justifica por el requisito del §4.5 y por criptas con más capacidad, no por el tamaño actual.
- `ubicar(id)` es lineal. Si los inventarios llegaran a miles de objetos y el replay fuera crítico, habría que añadir un índice por id con una tabla hash propia (no `dict`).
- El cursor no da la vuelta al llegar a un extremo. Si se pidiera un recorrido circular, solo cambian siguiente y anterior.
- Solo se equipan armas y armaduras. Si una clase nueva de objeto necesitara equiparse, habría que extender esa regla.

§4.6 Pergaminos de retroceso e historial reversible

Responsable: Integrante 3 — Kristel Peraza Sánchez, con los puntos de registro de Lógica hechos por el Integrante 1

1. Problema

Un pergamino de retroceso devuelve el juego al punto inmediatamente anterior a la última acción del jugador. Eso incluye todo lo que ocurrió después de esa acción y antes de la siguiente decisión: acciones enemigas, efectos, reloj, agenda de eventos, rastro, vidas, inventario, suelo, puertas y trampas. Se admiten como máximo 5 acciones hacia atrás y está prohibido guardar copias completas del estado. La cantidad de pergaminos queda fuera de la reversión, y la caché del catálogo también, porque es infraestructura y no estado del juego (§2.11).

Operaciones y frecuencia medida al reproducir partidas reales: en la partida de `cripta-03` (154 acciones) hubo 153 intervalos y 1 865 cambios registrados, una media de **12.2 cambios por intervalo** (mediana 10, máximo 29). En otras tres partidas reales la media fue de 11.5 a 12.6, con el mismo máximo de 29.

Operación	Cuándo ocurre	Frecuencia
Registrar un cambio	Cada mutación del estado (daño, movimiento, evento, recoger…)	La más alta: ~12 por acción
Cerrar el intervalo	Al terminar la acción del jugador y sus eventos	Una por acción
Reconstruir las fichas en uso	Tras cada acción y tras cada retroceso	Una por acción
Deshacer	Solo al leer un pergamino	Baja, pero debe ser exacta

2. Alternativas consideradas

- **Copia completa del estado por intervalo:** deshacer sería restaurar la copia. Prohibida por el §2.11 y el §4.6; además su costo crece con el tamaño del estado (actores, objetos, trampas, eventos y rastro), no con lo que cambió. Se mide igual en `--bench` para sustentar la decisión.
- **Un registro fijo por tipo de cambio:** datos simples y fáciles de inspeccionar. Descartada: el historial tendría que conocer cada tipo de cambio y su inverso, y cada regla nueva obligaría a modificarlo.
- **Una función de reversión por cambio (elegida):** cada cambio sabe cómo deshacerse; el historial solo las guarda y las ejecuta.

3. Decisión

`Logic/HistorialRetroceso.py`:

- **Función inversa por cambio.** Cada mutación entrega una función que la invierte. El historial guarda esas funciones agrupadas por intervalo; un intervalo es una acción del jugador más todo lo que ocurre hasta su siguiente decisión.
- **Buffer circular de 5 intervalos.** Al cerrar el sexto, el más antiguo se descarta sin desplazar nada.
- **Deshacer.** Ejecuta las funciones del último intervalo en orden inverso al registrado. Mientras deshace, los registros se ignoran; si no, el propio deshacer quedaría registrado.
- **Azar.** El estado del generador `random.Random(semilla)` se guarda una vez por intervalo, para que una acción repetida tras retroceder produzca la misma secuencia (§2.6).
- **Pergaminos y caché fuera del alcance.** La cantidad de pergaminos no se registra ni se restaura, y usar un pergamino no consume tiempo virtual (§2.11). La caché no se revierte; lo que sí se reconstruye tras cada acción y cada retroceso es la lista de fichas «en uso» (`SincronizadorUsos`), porque un retroceso puede revivir enemigos.

- **Quién registra.** El Motor posee el historial. El Integrante 1 agregó los puntos de registro en Lógica (agenda, rastro, mazmorra, actores y efectos) e hizo reversible el `Inventario`.
- **Servicios.** Leer un pergamino es la acción `retroceder`: no avanza el reloj y entra a la bitácora de replay para que la reproducción coincida. Con la partida terminada se rechaza toda acción, porque un pergamino ahí corrompería el estado (el último intervalo nunca se cerró).
- **Cuidado con las funciones creadas en un bucle.** En Python, una función creada dentro de un bucle ve la variable del bucle y no su valor de ese momento, así que todas compartirían el último. Por eso cada inversa se crea en una función auxiliar que recibe el valor (por ejemplo, `_cambiar_registrando` en la medición de `--bench`).

4. Análisis de complejidad

Sea k la cantidad de cambios de un intervalo (en partidas reales, ~12 de media y 29 como máximo) y N el tamaño total del estado.

Operación	Costo	Comentario
Registrar un cambio	O(1)	Se agrega una función al intervalo actual.
Cerrar el intervalo	O(1)	El buffer circular reemplaza el más antiguo sin desplazar.
Deshacer	O(k)	Cuesta lo que cambió, no el tamaño de la partida.
Memoria del historial	O(5 · k)	Acotada; no crece con la duración de la partida.
Copia completa (descartada)	O(N) por intervalo	Crece con actores, objetos, trampas, eventos y salas.

5. Evidencia empírica

Historial contra copia completa (grupo «historial» de `--bench`). Cada acción cambia 5 posiciones de un estado de N posiciones y luego se deshace. Costo por acción más su deshacer, en microsegundos:

Tamaño del estado (N)	Historial	Copia completa (descartada)
1 000	2.16 µs	1.88 µs
10 000	2.20 µs	14.6 µs
100 000	2.24 µs	277.7 µs

El historial cuesta lo mismo con cualquier tamaño de estado, porque solo depende de lo que cambió. La copia completa gana por poco con un estado de 1 000 posiciones, pero crece de

forma lineal: con 100 000 es unas 120 veces más lenta. Y esta copia es optimista, porque copia valores simples; con los objetos reales del juego costaría más. De todos modos sigue prohibida por el enunciado.

La corrección se verificó de extremo a extremo: `tests/test_retroceso_completo.py` hace 1 762 retrocesos en partidas completas y comprueba que el estado restaurado coincide con el anterior y que repetir la acción da el mismo resultado.

6. Límites conocidos

- Un intervalo con muchísimos cambios (cientos de enemigos activos con veneno y regeneración) hace crecer k, y con él la memoria y el costo de deshacer. Sigue siendo lineal en lo que cambió.
- Cada regla nueva que modifique el estado debe registrar su inversa. Un punto de registro olvidado produce un retroceso incompleto; se mitiga con las pruebas de extremo a extremo, pero el riesgo es propio de este diseño.
- Las funciones inversas mantienen referencias a objetos, así que un enemigo muerto sigue en memoria hasta que su intervalo sale del buffer (como máximo 5 intervalos).
- El límite de 5 lo fija el enunciado. Subirlo multiplica la memoria del historial.

§4.7 El rastro del jugador

Responsable: Integrante 1 — Marco Cabrera Mora (capa de Lógica)

1. Problema

Cada sala guarda el instante en que el jugador estuvo allí por última vez. En su turno, un rastreador se mueve a la sala vecina accesible con el rastro más fresco, siempre que tenga menos de 400 unidades de antigüedad; si empatan, elige la de menor id. Operaciones: **marcar** el rastro en cada acción del jugador (Apéndice B) y **consultarlo** una vez por cada salida abierta, en cada turno de cada rastreador. El §4.7 exige que la decisión cueste según las salidas de la sala y no según el tamaño de la cripta ni el recorrido del jugador, y que el trabajo total por ciclo no crezca más que linealmente con los enemigos que actúan.

2. Alternativas consideradas

- **Guardar el recorrido del jugador como una lista de visitas** y buscar en ella: el costo crecería con la longitud del recorrido, lo que prohíbe el §4.7. Descartada.
- **Borrar activamente los rastros viejos** con eventos programados en la agenda: cada marca programaría un borrado, lo que agrega eventos a la agenda y trabajo en cada acción y en cada retroceso. Descartada.
- **Array indexado por sala, con la caducidad detectada al consultar (elegida):** no se borra nada y la comparación `ahora - tiempo < 400` ya da la respuesta.

3. Decisión

`Logic/RastroSalas.py` y `Logic/Comportamientos.py`:

- **Patrón estrategia con NOMBRE y la lista COMPORTAMIENTOS.** Agregar un comportamiento es escribir una clase y sumarla a la lista, sin tocar la fábrica ni el motor.
- **El rastreador solo mira las salas vecinas abiertas.** Su costo depende de sus salidas (como máximo 4), no del tamaño de la cripta (§4.7).
- **El rastro es un array indexado por sala.** Consultarlo cuesta O(1).
- **La caducidad se detecta al consultar.** No se borran rastros viejos: no hace falta programar eventos de borrado ni recorrer nada, y comparar ahora − tiempo < 400 ya da la respuesta.
- **El rastro se marca en cada acción del jugador, no solo al moverse.** Así lo pide el Apéndice B.
- **Empate:** si dos vecinas tienen el mismo tiempo de rastro, se elige la de menor id (Apéndice B).
- **Integración con el retroceso:** cada marca se registra en el historial con su valor anterior, para que el pergamino restaure el rastro (§4.6).

4. Análisis de complejidad

- Marcar el rastro: **O(1)**, una posición del array.
- ¿Es fresco?: **O(1)**, una resta y una comparación.
- Decisión de un rastreador: **O(salidas de su sala)**, como máximo 4, sin importar el tamaño de la cripta ni el recorrido del jugador.
- Trabajo total por ciclo: **O(enemigos que actúan)**. Memoria: O(S), una posición por sala.

5. Evidencia empírica

`--bench` no aísla la decisión del rastreador; se mide dentro del ciclo de eventos (§4.1), donde un tercio de los enemigos sintéticos son rastreadores. Al multiplicar los enemigos activos, el tiempo del ciclo crece en la misma proporción, es decir, de forma lineal:

Enemigos activos	Eventos por ciclo	Mediana del ciclo	Crecimiento
10	~12	0.07 ms	—
100	~105	0.82 ms	×10 enemigos → ×12 tiempo
500	~521	4.35 ms	×5 enemigos → ×5.3 tiempo

En partidas reales en `cripta-01` las ratas gigantes (rastreadoras) siguen al jugador cuando sale de su sala ("Rata gigante #1 entra a la sala"), dentro del ciclo normal.

6. Límites conocidos

- El array usa el id de sala como posición, así que supone ids seguidos (1..N), como los del servicio; con ids muy dispersos se desperdiciaría memoria.

- Como nada se borra, el array guarda una posición por sala aunque muchas nunca se visiten (O(S) de memoria, poco incluso con miles de salas).
- La evidencia del rastro es indirecta: el bench mide el ciclo completo, no la decisión del rastreador sola.

§4.8 Bitácora en pantalla

Responsable: Integrante 3 — Kristel Peraza Sánchez

1. Problema

La interfaz muestra solo los últimos 20 mensajes del juego. La memoria de esa bitácora debe ser constante durante toda la partida, y agregar un mensaje no puede desplazar físicamente los 19 anteriores (§4.8). Operaciones: **agregar** un mensaje por cada evento relevante (ataques, daños, muertes, avisos) y **leer los 20 en orden** al armar el estado que se entrega a la Vista, una vez por acción. En la partida real de `cripta-03` se agregaron 230 mensajes en 154 acciones (1.5 por acción), con picos en los combates.

2. Alternativas consideradas

- **Arreglo al que se le quita el primer elemento:** implementación inmediata. Descartada: cada mensaje nuevo desplaza los otros 19, justo lo que prohíbe el §4.8. Se mide en `--bench`.
- **collections.deque con tope:** ya hecha. Descartada: es una estructura de biblioteca y el §7 prohíbe sustituir con ellas las estructuras propias.
- **Buffer circular propio (elegida):** memoria fija y agregar sin mover nada.

3. Decisión

`Estructuras/BufferCircular.py`: un arreglo de tamaño fijo y un índice de escritura que vuelve al inicio al llegar al final. Al llenarse, el mensaje nuevo sobrescribe el más antiguo, y la lectura entrega los mensajes del más antiguo al más reciente. Se quitaron dos métodos que nada usaba. El mismo principio sostiene el historial de 5 intervalos del §4.6.

4. Análisis de complejidad

Sea C la capacidad (20).

Operación	Costo	Comentario
Agregar un mensaje	O(1)	Se escribe en la posición actual y avanza el índice.
Leer los 20 en orden	O(C)	C es constante, así que la lectura tiene costo fijo.
Memoria	O(C)	Constante durante toda la partida.

Operación	Costo	Comentario
Arreglo que desplaza (descartado)	O(C) por mensaje	Mueve C − 1 elementos cada vez.

5. Evidencia empírica

Buffer circular contra el arreglo que desplaza (grupo «bitácora en pantalla» de `--bench`). Se agregan muchos más mensajes que la capacidad y se comprueba que quedan los últimos C. Costo por mensaje agregado, en microsegundos:

Capacidad (C)	Buffer circular	Arreglo que desplaza (descartado)
20	0.09 µs	0.07 µs
200	0.10 µs	0.07 µs
2 000	0.11 µs	0.15 µs
10 000	0.11 µs	0.57 µs

Con la capacidad real de 20, el arreglo que desplaza es un poco más rápido: mover 19 referencias lo hace Python en una sola operación de memoria. La diferencia es de centésimas de microsegundo por mensaje, irrelevante frente a los 20 ms del ciclo (§5.2). El costo del buffer no depende de C, mientras que el del arreglo crece con C: con 10 000 ya es cinco veces más lento. La decisión se apoya en el requisito literal del §4.8 (no desplazar) y en ese comportamiento, no en la velocidad con C = 20.

6. Límites conocidos

- La capacidad de 20 la fija el enunciado. Si se pidiera un historial navegable de miles de mensajes, haría falta otra estructura.
- Los mensajes antiguos se pierden por diseño. La bitácora de replay guarda las acciones del jugador, no los mensajes.
- La Vista ajusta los mensajes largos al ancho del panel (350 px) y corta los avisos muy largos a 3 líneas.

§4.9 Ordenamiento

Responsable: Integrante 3 — Kristel Peraza Sánchez

1. Problema

El programa ordena en varios contextos: la tabla de puntajes (en Servicios, por omisión por enemigos derrotados), las vistas del inventario por peso, valor o nombre, los índices del archivo de guardado y listas auxiliares del catálogo. Los tamaños, el grado de desorden y el costo de comparar varían: comparar números es barato y comparar nombres sin distinguir

mayúsculas es más costoso. El enunciado pide al menos dos algoritmos con características distintas, elegir de forma automática según propiedades medibles de los datos y sustentar la elección con mediciones propias, y prohíbe `list.sort()` y `sorted()`.

En el juego, la vista del inventario se arma una vez por acción, y cuando el jugador eligió un orden se ordenan como máximo `inventario_max` objetos (10 en las criptas del servicio). La tabla de puntajes crece una fila por partida terminada.

2. Alternativas consideradas

- **Un solo algoritmo:** menos código. Descartada: solo mezcla desperdicia tiempo y memoria con listas pequeñas, y solo inserción es cuadrática cuando la lista crece. Se mide cada uno forzado.
- **Quicksort:** rápido en promedio y en sitio. Descartado: no es estable (con empates el resultado cambiaría según el algoritmo elegido) y su peor caso es cuadrático.
- **Inserción y mezcla (elegida):** dos comportamientos opuestos, y ambos estables.

3. Decisión

`Estructuras/Ordenador.py`, en `Estructuras/` porque lo usan Lógica, Servicios y Datos:

- **Dos algoritmos estables.** Inserción (sin memoria extra, excelente con pocos datos o casi ordenados) y mezcla (O(n log n) siempre, con un arreglo auxiliar). Como los dos son estables, el resultado es idéntico sin importar cuál se elija; la elección solo afecta el tiempo.
- **Interfaz.** `ordenar(lista, comparador)`: el comparador devuelve un entero y la función entrega una lista nueva sin modificar la original.
- **Elección automática** según el tamaño n y el desorden, medido como la proporción de pares vecinos fuera de orden (cuesta n − 1 comparaciones): si n ≤ 20 se usa inserción sin medir el desorden; si 20 < n ≤ 128 y el desorden es ≤ 0.25, inserción; en cualquier otro caso, mezcla. El tope de 128 acota el peor caso cuadrático de la inserción.
- **Umbral corregido por la medición.** El umbral pequeño empezó en 12. El bench mostró que con 13 a 20 elementos desordenados la elección automática elegía mezcla y perdía contra la inserción, porque además pagaba la medición del desorden; se subió a 20.
- **Sin sort() ni sorted()** en ningún ordenamiento del programa.

4. Análisis de complejidad

Algoritmo	Mejor	Promedio	Peor	Memoria extra	Estable
Inserción	O(n)	O(n²)	O(n²)	O(1)	Sí
Mezcla	O(n log n)	O(n log n)	O(n log n)	O(n)	Sí

Como se entrega una lista nueva, los dos copian la entrada en O(n). Medir el desorden cuesta O(n) y solo se hace entre 21 y 128 elementos. Las ordenaciones más frecuentes del juego

son listas pequeñas (la vista del inventario), donde la inserción gana por su bajo costo constante; la mezcla se reserva para tablas grandes, donde evita el peor caso cuadrático.

5. Evidencia empírica

Grupo «ordenamiento» de `--bench`: cada algoritmo forzado y la elección automática, con datos desordenados (comparador barato: números; costoso: texto sin distinguir mayúsculas), casi ordenados y dos tramos ordenados puestos al revés. Microsegundos por ordenamiento:

Escenario	n	Inserción	Mezcla	Automático (elige)
Desordenados, comparador barato	8	1.9 µs	4.2 µs	2.0 µs (inserción)
Desordenados, comparador barato	16	6.7 µs	9.7 µs	7.0 µs (inserción)
Desordenados, comparador barato	24	14.5 µs	19.5 µs	21.3 µs (mezcla)
Desordenados, comparador barato	32	23.2 µs	26.2 µs	29.2 µs (mezcla)
Desordenados, comparador barato	64	107.8 µs	61.6 µs	68.7 µs (mezcla)
Desordenados, comparador barato	128	434.8 µs	145.6 µs	164.5 µs (mezcla)
Desordenados, comparador costoso	16	14.6 µs	15.9 µs	14.8 µs (inserción)
Desordenados, comparador costoso	24	34.4 µs	31.7 µs	37.8 µs (mezcla)
Desordenados, comparador costoso	128	976.9 µs	244.6 µs	270.9 µs (mezcla)
Casi ordenados (2 % de pares cambiados)	128	13.1 µs	29.2 µs	21.6 µs (inserción)
Casi ordenados (2 % de pares cambiados)	2 048	233.0 µs	530.0 µs	521.0 µs (mezcla)
Dos tramos ordenados al revés	128	433.0 µs	49.0 µs	430.0 µs (inserción)
Dos tramos ordenados al revés	512	6 764 µs	173.0 µs	172.0 µs (mezcla)

Lo que muestra la medición: con datos desordenados, la inserción gana hasta 32 elementos con comparador barato y hasta 16 con costoso; a partir de 64 la mezcla es de 1.7 a 4 veces más rápida. Con datos casi ordenados la inserción es unas 2 veces más rápida hasta 128

elementos. La elección automática nunca usa inserción donde esta pierde por mucho en datos desordenados, y por encima de 128 siempre usa mezcla, lo que evita el caso de 512 elementos en dos tramos al revés, donde la inserción es 39 veces más lenta.

6. Límites conocidos

- Los umbrales (20, 0.25 y 128) se calibraron en las computadoras del equipo. En otra máquina el cruce puede moverse; por eso `--bench` permite repetir la medición. En esta corrida, con comparador barato, la inserción todavía ganaba con 24 y 32 elementos, donde la elección automática usa mezcla y pierde de 6 a 7 µs por ordenamiento: el umbral de 20 es conservador.
- El desorden se mide con pares vecinos, así que no detecta dos tramos ya ordenados pero puestos al revés (desorden casi 0 y muchas inversiones). Con 128 elementos la elección automática usa inserción y tarda 430 µs contra 49 µs de la mezcla; el tope de 128 acota ese caso a menos de medio milisegundo.
- Con comparadores mucho más costosos que los medidos, el cruce baja y 20 podría quedar alto.
- Que ambos algoritmos sean estables es una propiedad que hay que conservar: si se añadiera uno inestable, la elección automática dejaría de ser invisible en el resultado.

§4.10 Guardado binario de la partida

Responsable: Integrante 2 — Derek García Herra (capa de Datos)

1. Problema

Guardar el estado completo en un binario de formato propio, poder recuperar una sala sin leer todo el archivo, detectar otra versión del formato y comparar tamaño y tiempo contra JSON. Guardar y cargar son poco frecuentes (cuando el jugador lo pide), pero el estado incluye todas las salas y crece con la cripta.

2. Alternativas consideradas

- **JSON con los mismos DTOs:** legible, pero ~3 veces más grande y obliga a leer todo para obtener una sala (comparación exigida y medida).
- **Binario con índice ordenado y búsqueda binaria:** O(log n) lecturas por sala. Descartada.
- **Binario con índice de acceso directo (elegida):** la posición de la sala en el índice se calcula.

3. Decisión

`Data/GuardadoBinario.py` (`struct`, little-endian): cabecera fija de 34 bytes (marca `CRPT`, versión del formato, posiciones del bloque global y del índice, id mínimo), bloque global, un bloque por sala e índice de 12 bytes por sala; la sala `id` está en `offset_indice + (id − id_minimo) × 12`. La marca y la versión se revisan antes de leer nada más, y cualquier archivo

que no cuadre lanza `ErrorGuardado`. Se escribe a `.tmp` y luego reemplaza al archivo, así que una caída no daña el guardado anterior.

Acordado con el equipo: se guardan **todas** las salas (puertas, enemigos que se mueven y botín pueden afectar salas sin contenido cargado), solo los eventos válidos con su dueño como referencia, el inventario por posiciones, el estado completo del azar y la ruta de la bitácora. El juego reconstruye todas las salas al cargar (`cargar_salas`); `cargar_sala` cumple la propiedad de leer una sola.

4. Análisis de complejidad

Guardar **O(estado + R)**; cargar una sala **O(1) lecturas** + O(tamaño de la sala), sin importar cuántas salas haya; cargar todo **O(archivo)** en una sola lectura (R = rango de ids, igual a S si son seguidos).

5. Evidencia empírica

Salas	Tamaño bin / JSON	Guardar	Cargar todo	Cargar 1 sala
20	5.1 / 15.0 KB	0.60 / 0.90	0.40 / 0.29	0.093 / 0.215
200	20.7 / 62.1 KB	1.21 / 3.83	1.50 / 1.12	0.097 / 0.575
2 000	168.7 / 500.4 KB	5.93 / 32.30	11.96 / 9.66	0.105 / 4.560

Tiempos en ms, binario / JSON. El binario ocupa ~3 veces menos, guarda hasta 5.4 veces más rápido y carga una sala en tiempo casi constante (43 veces más rápido que JSON con 2 000 salas). Cargar todo es algo más lento que JSON (12.0 contra 9.7 ms), porque el lector de `json` está escrito en C y nuestro decodificador corre en Python; para acercarlo se precompilaron los formatos y el archivo se lee de una vez. En una partida real de `cripta-01`, el guardado ocupó ~3 KB y la partida cargada siguió con el mismo reloj.

6. Límites conocidos

- El índice supone ids seguidos: cada hueco cuesta 12 bytes (hay un límite de un millón de entradas).
- Cargar todo es lo primero que se notaría con criptas mucho más grandes.
- No hay migración entre versiones del formato: los archivos viejos se rechazan con un mensaje claro.
- El historial de retroceso no se guarda: después de cargar, el pergamino solo deshace acciones posteriores a la carga.

§4.11 Bitácora de partida y reproducción

Responsable: Integrante 2 — Derek García Herra (capa de Datos)

1. Problema

Registrar cada acción del jugador en un texto de solo anexado, con un encabezado (id y versión de la cripta, versión del catálogo y semilla), y reproducir la partida con `--replay` sin interacción, avisando si la versión ya no está disponible. Se anexa una línea **por cada acción** y el log completo se lee una vez por reproducción.

2. Alternativas consideradas

- **Guardar las acciones en memoria y escribir al final, o reescribir el archivo en cada acción:** se pierde la partida si el programa se cierra, o cuesta O(N) por acción. Descartada.
- **Formato binario:** más chico, pero el log es la herramienta para reportar errores en la evaluación y debe ser legible. Descartada.
- **Texto, una línea por acción, en modo de anexado (elegida).**

3. Decisión

`Data/BitacoraReplayIO.py`: `#CRIPTA-REPLAY 1`, las líneas `id_cripta=`, `version_cripta=`, `version_catalogo=`, `semilla=`, luego `#ACCIONES` y una acción por línea (`tipo|dirección|objetivo|objeto`). Nunca sobrescribe (cada partida usa el siguiente `logs/partida_NNN.log`). Antes de escribir comprueba que la línea se lee como la misma acción; al leer, una última línea incompleta se descarta con aviso; si las versiones no coinciden, se informa y no se reproduce. Al cargar un guardado, `continuar_desde` copia las acciones previas a un log nuevo, que se puede reproducir completo. El uso del pergamino también queda registrado.

4. Análisis de complejidad

Anexar **O(largo de la línea)**, constante respecto al tamaño del log; leer **O(N)**; `continuar_desde` O(N) una vez al cargar.

5. Evidencia empírica

Acciones en el log	Anexar 1 acción	Leer todo
100	0.133 ms	0.25 ms
1 000	0.136 ms	1.43 ms
10 000	0.149 ms	14.0 ms

- Anexar es constante y leer crece linealmente, como se esperaba.
- **Partida real en cripta-02:** 83 acciones (incluido un pergamino) reproducidas con el mismo final: **victoria con el reloj en 5625**.
- **Partida cargada en cripta-01:** su log (10 acciones copiadas más 1 nueva) se reprodujo hasta la misma derrota (reloj 905) que registró la tabla de puntajes.

6. Límites conocidos

- Cada anexado abre y cierra el archivo (~0.13 ms): despreciable por acción del jugador, pero no para miles de eventos por segundo.
- Reproducir en línea vuelve a pedir la cripta (gasta solicitudes), y si la cripta cambió de versión el replay se rechaza, como exige el enunciado.

Anexo A: otras decisiones de la capa de Lógica

Responsable: Integrante 1 — Marco Cabrera Mora (capa de Lógica)

Decisiones de la capa de Lógica que no corresponden a un solo problema del §4. Quedan fuera HistorialRetroceso.py e Inventario.py, que son del integrante 3; solo se menciona cómo Lógica se conecta con ellos. PartidaGuardada.py es compartido entre los tres integrantes, así que se incluye solo su parte de Lógica.

Actores (ActorBase.py, jugador.py, Enemigo.py)

- **Herencia ActorBase → Jugador / Enemigo solo en Lógica.** Ahí está el comportamiento compartido (daño, curación, intervalo) y el distinto (equipo en el jugador; ficha y estrategia en el enemigo).
- **El actor no hereda de ActorDTO.** El DTO es frozen y el actor cambia en cada ciclo. Además, la Vista no debe recibir objetos vivos.
- **a_dto() en la base.** Una sola conversión para jugador y enemigo. Funciona porque cada subclase define tipo() y nombre_visible(), así nadie pregunta con isinstance.
- **Separar ficha e instancia.** Una ficha del catálogo sirve para muchas instancias (relación uno a muchos), así que el enemigo guarda una referencia a la ficha en lugar de copiar sus datos. El jugador se construye con CriptaDTO, porque es un solo jugador por cripta y no necesita ficha.
- **ataque_efectivo() y defensa_efectiva().** El combate llama siempre al mismo método, y Jugador lo sobrescribe para sumar el bonus del equipo.
- **Vida None → vida_max de la ficha.** Es lo que indica el contenido del API cuando no trae la vida.

Mazmorra (Mazmorra.py)

- **Estado vivo separado del DTO.** Los SalaDTO describen el estado inicial y son inmutables. Las clases SalaViva, SalidaViva y TrampaViva guardan lo que cambia durante la partida.
- **Salas en un array indexado por id.** Acceso O(1) sin dict.
- **Salidas en un array de 4 (N, S, E, O).** Es la convención del grupo, y fija el orden en que el errante recorre las salidas.
- **Cada sala lleva su lista de enemigos.** Para saber quién está con el jugador basta revisar esa sala, sin recorrer todos los enemigos de la cripta.

- **Puertas de dos caras.** Cada salida conecta dos salas (§2.1), así que abrir una cara abre también la inversa. La cripta declara la llave y el cierre automático en una sola cara; al cargar el esqueleto se copian a la cara de vuelta (_igualar_caras), para que una puerta que se cerró sola se pueda volver a abrir desde el otro lado (Apéndice B).
- **El cierre automático es un evento de la agenda.** Reutiliza el mismo mecanismo de tiempo, sin contadores aparte.
- **Numeración "#n" con un array de pares [tipo, cantidad].** No se puede usar dict, y con pocos tipos por cripta la búsqueda lineal es suficiente. El número se asigna una sola vez, para que no cambie cuando muere otro enemigo del mismo tipo.

Motor (motorJuego.py)

- **Una sola instancia de random.Random(semilla).** Garantiza que la misma semilla produzca la misma partida, que es lo que necesita el replay.
- **avanzar_hasta_jugador() se detiene en cada decisión del jugador.** El motor no sabe si quien decide es la consola, una interfaz gráfica o el replay, así que pasar a otra interfaz no toca Lógica.
- **Todo evento llama a dueno.activar(motor, evento).** Es el principio abierto/cerrado: un tipo de evento nuevo es una clase con activar, sin agregar if al motor.
- **Las acciones devuelven un código** ("ok", "cerrada", "sin_llave"...). Si se rechazan, no consumen tiempo (Apéndice B), y Servicios decide qué mensaje mostrar.
- **"pendiente" cuando falta el contenido de la sala destino.** Lógica no conoce la red: Servicios carga el contenido y reintenta, y el reloj no avanza mientras tanto (§2.4).
- **Dependencias inyectadas** (fn_mensaje, fn_tiene_objeto, fn_ficha). El motor no conoce la bitácora, la caché ni la lista enlazada de los otros integrantes.
- **cambiar_velocidad escala el tiempo restante.** Aplica la fórmula del §2.3 y vuelve a programar el evento para que reciba una secuencia nueva.

Efectos (Efectos.py, GestorEfectos.py)

- **Clase base Efecto con al_aplicar y activar, y un aplicar() genérico en el gestor.** Abierto/cerrado otra vez: la regeneración se agregó después sin modificar el gestor.
- **UNICO para veneno y regeneración.** Si se aplica de nuevo, reemplaza al anterior en vez de acumularse.
- **Velocidad guarda cambio_real.** Al terminar revierte exactamente lo que cambió, aunque la velocidad haya quedado topada en 1.
- **cancelar_de(actor) al morir.** Solo recorre la lista de efectos activos, no la agenda (§4.1).
- **visibles() entrega EfectoDTO con el tiempo restante.** La Vista no ve el instante en que termina cada efecto.

Integración con retroceso y guardado

- **Cada cambio se registra en el historial antes de hacerse.** El enunciado prohíbe guardar copias del estado. Se registran también la extracción de eventos de la cola y el estado del azar en cada intervalo, para que después de retroceder el juego siga siendo determinista.
- **Los cambios de preparación se descartan, y los pergaminos no son reversibles.** Así lo indica el §2.11.
- **PartidaGuardada guarda los eventos válidos en orden y con sus secuencias.** Al cargar, el montículo queda igual y los desempates no cambian. El historial no se guarda. Es un archivo compartido entre los tres integrantes: el 1 aporta el estado de Lógica, el 2 el guardado binario y el 3 el servicio que lo orquesta.

Criterios generales

- **KISS y YAGNI.** Se dejó fuera lo que nada usaba todavía; por ejemplo, la regeneración entró solo cuando el catálogo la declaró.
- **Restricciones del §7.** Nada de dict, sorted() ni heapq. Para ordenar se usa el Ordenador del equipo.
- **Estilo del equipo.** Los nombres de atributos quedaron sin prefijos de tipo, para mantener la convención de todo el proyecto.

Anexo B: decisiones de Servicios, Controlador y Vista

Responsable: Integrante 3 — Kristel Peraza Sánchez

Estas decisiones no corresponden a un problema del §4, pero sostienen la arquitectura por capas del §6.

Servicios

Tema	Decisión	Por qué
Estado de la partida	Servicios sin estado y un `ContextoPartida` que se les pasa: `ejecutar(contexto, accion)`.	Se prueban fácil, sin preparar objetos ocultos.
Clases auxiliares	`ArmadorEstado`, `SincronizadorUsos` y `ContextoPartida`.	Cada una la usan al menos dos servicios; sin ellas se duplicaría código.
Mover	Datos primero y Lógica después: se carga la sala destino antes de llamar al motor.	Si falla la red, el juego queda intacto.

Tema	Decisión	Por qué
Validar la acción	Servicios rechaza direcciones inválidas y datos faltantes.	La bitácora de replay no pasa por el Controlador, y las pruebas mostraron que esas guardas hacen falta.
Qué entra a la bitácora	Solo las acciones que se hicieron, incluido retroceder; las rechazadas no.	Una acción rechazada no consume tiempo.
Replay ante datos distintos	`ErrorReplay` sin ejecutar nada si las versiones no coinciden o la partida diverge.	§4.11: nunca reproducir en silencio contra datos distintos.
Cargar partida	No se guardan historial, mensajes ni nombre. Se comprueban las versiones, y la bitácora continúa en un archivo nuevo al final de la carga.	Una partida sin replay rompería la garantía del §4.11.
Puntajes	Datos guarda las filas; Servicios las ordena con el `Ordenador`, por omisión por enemigos derrotados.	El puntaje no indica si se ganó, y ordenar por acciones premiaría morir rápido.
Estado para la Vista	Copia de la sala con las puertas y el suelo actuales y sin enemigos ni trampas; los enemigos se envían aparte, desde el estado vivo de Lógica.	El estado vivo es de Lógica; la sala inicial mostraría enemigos con la vida completa.

Controlador y Vista

Tema	Decisión	Por qué
Controlador	Orquesta entre la Vista y Servicios. Un error de un servicio muestra su mensaje sin cerrar la sesión.	Es la arquitectura del curso (MVC por capas).
Acceso a Servicios	Un único punto, `ControladorBackend`, con los errores esperados unificados en `ErrorOperacion`.	La interfaz nunca llama a Datos ni a Lógica.
División del Controlador	`ControladorPygame` lleva el flujo de pantallas (menú, pausa, fin) y `ControlesJuego` traduce teclado y ratón durante la partida.	Cambian por razones distintas y se prueban por separado.

Tema	Decisión	Por qué
Selección de objetos	Enemigos y objetos del suelo se eligen con el ratón o el teclado; soltar, equipar y usar actúan sobre el objeto del cursor.	§4.5: se suelta «el objeto que se está viendo».
Modos sin interfaz	`--replay` y `--bench` llaman directo a Servicios; pygame y la Vista se importan solo dentro de `jugar()`.	§6: esos modos no necesitan la interfaz; una prueba en un intérprete limpio lo verifica.
Biblioteca gráfica	`pygame-ce`, que se usa con el mismo `import pygame`.	Instala sin problemas en las versiones de Python del equipo (3.11 a 3.14).
Contrato de la Vista	Cuatro métodos: `abrir`, `dibujar(PantallaDTO)`, `cerrar` y `elemento_en`.	Es lo mínimo; el Controlador no conoce pygame.
Estructuras en la Vista	Sin `dict`, `set`, `sorted` ni `.sort(` en `View/`; las tablas son listas de tuplas.	§7 y separación MVC.
Funciones opcionales	Animaciones, sonido y la demora al terminar están apagadas por omisión en la Vista y el Controlador; `main.py` las activa.	Con los valores por omisión la salida no cambia y las pruebas anteriores siguen valiendo.
Entrada de la Vista	La Vista no lee el ratón: el elemento bajo el ratón y las activaciones viajan en el `PantallaDTO`.	Mantiene la Vista sin entrada propia.
Sin datos de una cripta	Los sprites se buscan por id del catálogo, con respaldo por clase y un dibujo genérico.	§9: la revisión usa una cripta nueva y un sprite ausente no debe romper el juego.
Presupuesto de dibujo	30 cuadros por segundo; media de dibujo < 12 ms y máximo < 25 ms (medido: ~10.7 ms de media con todo activo).	Hay 33 ms por cuadro. Es aparte del límite de 20 ms del §5.2, que excluye el dibujo.

Anexo C: reglas que el enunciado no define

Responsable: Integrante 3 — Kristel Peraza Sánchez

El enunciado deja algunos comportamientos sin especificar. Para cada caso se fijó una sola regla, se documentó aquí y se probó.

Caso	Regla adoptada
Dónde se recoge un objeto	Al final del inventario.

Caso	Regla adoptada
Cursor en los extremos	No da la vuelta.
Cursor tras soltar	Pasa al siguiente objeto; si no hay, al anterior.
Equipar	El objeto sube al frente y el cursor lo sigue. Solo se equipan armas y armaduras; la sustituida se queda en el inventario, y soltar un objeto equipado lo desequipa.
Varios objetos con el mismo id	El equipo se marca por nodo y `ubicar(id)` prefiere la instancia de repuesto.
Orden por omisión de los puntajes	Por enemigos derrotados.
Soltar una antorcha con otra encendida	Usar una antorcha la consume y deja el efecto encendido durante su duración (§2.7). El enunciado no menciona qué pasa al soltar otra antorcha mientras una está encendida; el equipo decidió que soltarla apaga la encendida. Está en `Logic/motorJuego.py` y la prueba `tests/test_antorcha_soltar.py`.

Observación sobre la dificultad

En `cripta-01`, dos ratas gigantes matan al jugador en todas las partidas en que pelea contra ellas (300 semillas probadas). No es un error: son las reglas del §2 (velocidad 150, y la primera mordida ocurre antes de la primera decisión del jugador). No se modificaron reglas para hacerla más fácil.